

Lecture 11

게임 배포: Microgame부터 GitHub Pages까지

Unity를 이용한 2D 게임프로그래밍

강영민

동명대학교 게임공학과

2026년 1학기

목차

- 1 학습목표
- 2 Part 1. Unity Microgame
- 3 Part 2. WebGL로 Export 하기
- 4 Part 3. GitHub Pages로 배포하기
- 5 정리 및 과제

학습목표

이번 강의에서 배울 내용

- Unity가 제공하는 **Microgame** 템플릿을 이용해 완성된 게임 프로젝트를 빠르게 체험한다.
- Unity 프로젝트를 **WebGL**로 빌드(Export)하여 웹 브라우저에서 실행 가능한 형태로 만든다.
- 빌드 결과물을 **GitHub Pages**에 올려 누구나 접속할 수 있는 **공개 URL**로 배포한다.

완성 목표

내가 만든(또는 수정한) Unity 게임을

<https://<나의계정>.github.io/<프로젝트>/> 형태의 URL로 친구에게 공유할 수 있다.

전체 배포 파이프라인 개요



이번 강의의 흐름

- 1 **Microgame**으로 빠르게 돌아가는 게임을 확보
- 2 Unity의 **Build Settings**에서 WebGL 빌드 산출
- 3 빌드 산출물을 GitHub 저장소에 올리고 **Pages**로 배포

Unity Microgame이란?

Microgame이란

Unity가 초보자 학습용으로 제공하는 완성된 형태의 작은 게임 프로젝트 템플릿이다.

- 게임 로직, 스크립트, 에셋, 씬이 모두 포함
- 실행하면 바로 플레이 가능
- 단계별 모드(Mod) 튜토리얼로 간단한 수정을 통해 게임을 변형하며 배움

이 강의에서 사용하는 이유

- 빌드/배포에 집중하기 위해
“돌아가는 게임”을 빠르게 확보
- 본인이 만든 프로젝트로 대체해도 좋음

대표적인 Microgame 종류

- 2D Platformer Microgame
- Karting Microgame
- FPS Microgame
- LEGO Microgame
- John Lemon's Haunted Jaunt

Unity 버전에 따라 제공 목록이 달라질 수 있으므로, **Unity Hub** – New project 화면에서 직접 확인.

Microgame 프로젝트 생성 (Unity Hub)

절차 – Unity Hub

- 1 **Unity Hub** 실행
- 2 왼쪽 **Projects** 탭 → **New project**
- 3 상단 **Editor Version**이 LTS 버전인지 확인
- 4 좌측 카테고리에서 **Learning** (또는 Sample) 선택
- 5 원하는 **Microgame** 템플릿 선택
(예: 2D Platformer Microgame)
- 6 **Project name, Location** 지정
- 7 **Create project** 클릭

버전/카테고리가 보이지 않을 때

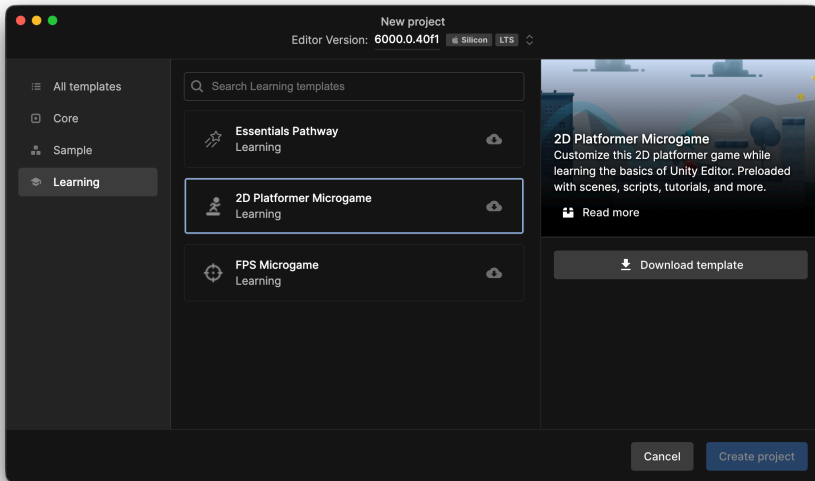
Unity Hub 버전에 따라 Microgame이 **Learning** 탭에 보이지 않을 수 있다.

- Unity Hub를 최신 버전으로 업데이트
- **Asset Store / Unity Learn** 사이트에서 해당 프로젝트를 **Open in Unity Hub** 버튼으로 등록

본인 프로젝트 사용 시

앞 시간에 만든 **Snowboard, Arcade** 게임 등 이미 가지고 있는 **Unity** 프로젝트로 대체 가능.

Unity Hub – Microgame 선택 화면



Microgame 프로젝트 구조 살펴보기

Project 폴더 예 (2D Platformer)

- ▽ Assets
 - Scenes/
 - Scripts/
 - Prefabs/
 - Sprites/
 - Audio/
 - Tutorial Info/
- ▽ Packages
- ▽ ProjectSettings

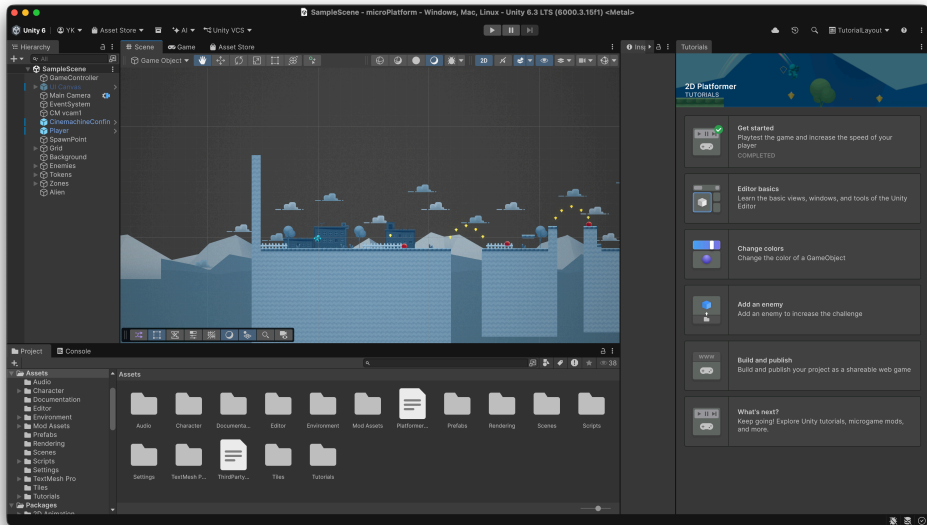
바로 해보기

- 1 Project 창에서 **Scenes** 폴더의 메인 씬을 더블 클릭하여 열기
- 2 상단 **Play** 버튼(▶) 클릭
- 3 **Game View**에서 키보드/마우스로 플레이
- 4 다시 **Play** 버튼을 눌러 정지

주의

Play 상태에서 변경한 값은 저장되지 않는다. 먼저 정지한 뒤에 수정할 것.

Microgame – 에디터 첫 화면



(선택) 간단히 게임 수정해 보기

쉬운 수정 아이디어

- Player의 이동 속도/점프 힘 값 변경
- 적/장애물의 개수나 색상 변경
- 배경 스프라이트 교체
- UI 텍스트("Game Start" 등) 문구 변경

목적

“내가 만든 게임처럼 보이도록” 살짝 수정한 뒤 WebGL 빌드로 넘어간다.

수정 후 반드시 할 일

- ① **Ctrl/Cmd + S** – 싼 저장
- ② **File → Save Project**
- ③ **Play**로 결과 다시 확인

싼 저장을 잊으면

에디터에서 수정한 내용이 빌드에 반영되지 않을 수 있다. WebGL 빌드 전에 반드시 싼 저장.

WebGL이란?

WebGL의 개념

WebGL(Web Graphics Library)은 브라우저에서 플러그인 없이 GPU 가속 그래픽을 그릴 수 있는 표준이다.

- OpenGL ES 기반
- 모든 주요 브라우저에서 지원 (Chrome, Edge, Firefox, Safari)
- Unity는 게임을 **JavaScript + WebAssembly**로 컴파일하여 WebGL 위에서 실행

Unity WebGL 빌드의 장점

- **설치 불필요** – URL 하나로 실행
- **크로스 플랫폼** (PC/Mac/모바일 브라우저)
- **공유 용이** – 친구, 포트폴리오, 채용 담당자

한계

- **모바일 성능 제약**
- 일부 **C# 기능** (멀티 스레딩 등) 제한
- 빌드 크기/로딩 시간 큼

WebGL Build Support 모듈 설치

Unity Hub에서 모듈 설치

- 1 Unity Hub → **Installs** 탭
- 2 사용 중인 **Editor** 버전 우측 : 메뉴 클릭
- 3 **Add modules** 선택
- 4 **Platforms** 항목에서
WebGL Build Support 체크
- 5 **Install** 클릭, 다운로드 대기

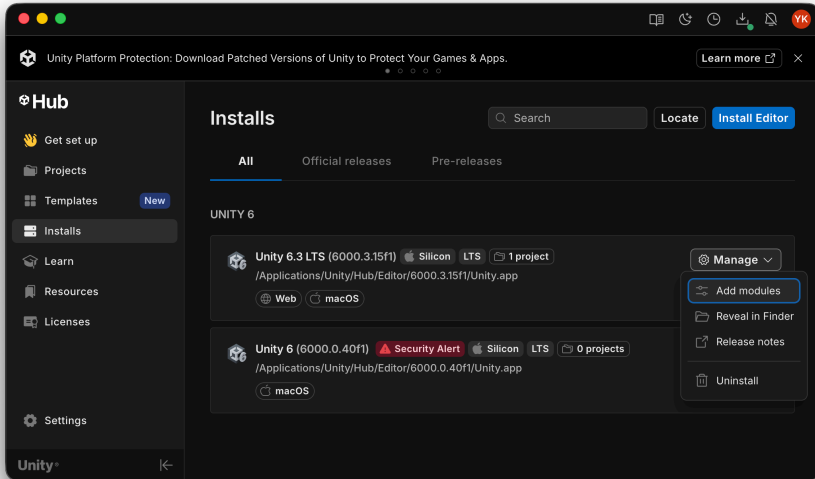
이미 프로젝트를 열어둔 경우

설치가 끝난 뒤 **Unity Editor**를 재시작해야 File → Build Settings에서 **WebGL**이 선택 가능 상태로 표시된다.

디스크 공간

WebGL 모듈은 약 수 **GB** 용량을 차지한다. 설치 전 충분한 여유 공간 확인.

Unity Hub – Add modules



Build Settings – 플랫폼 전환

플랫폼을 WebGL로 변경

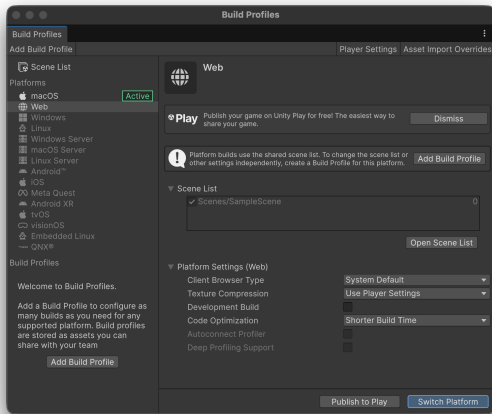
- 1 **File** → **Build Settings**
(단축키: Ctrl/Cmd + Shift + B)
- 2 **Platform** 목록에서 **WebGL** 선택
- 3 우측 하단 **Switch Platform** 클릭
- 4 에셋이 재임포트된다 (수 분 소요 가능)
- 5 변환이 끝나면 **Unity** 로고가 WebGL 옆으로 이동

Scenes In Build

Build Settings 상단의
Scenes In Build 목록 확인.

- 빌드에 포함할 씬에 체크
- 빈 경우 현재 열린 씬을
Add Open Scenes 버튼으로 추가
- **Index 0**이 첫 실행 씬

Build Profiles – Web 플랫폼 선택



Platforms 목록에서 **Web**을 선택한 뒤 우측 하단 **Switch Platform** 클릭. **Scene List**와 **Platform Settings**도 함께 확인한다.

Player Settings – 기본 정보

경로

Build Settings 창 하단 **Player Settings** 버튼
또는 **Edit** → **Project Settings** → **Player**

꼭 확인할 항목

- **Company Name, Product Name**
– 브라우저 탭 제목/폴더 이름 기준
- **Default Icon** (선택)
- 좌측 탭 중 **WebGL** 아이콘 선택

Resolution and Presentation

WebGL 탭에서 설정.

- **Default Canvas Width / Height**
예: 960 × 600
- **Run In Background:** 보통 ON
- **WebGL Template:**
Default 또는 Minimal

Player Settings – Publishing Settings (중요)

Compression Format

WebGL 빌드 결과는 기본적으로 **Brotli** 압축이지만, GitHub Pages 같은 **정적 호스팅**은 Brotli/Gzip 헤더를 자동으로 붙여주지 않을 수 있다.

권장: Compression Format = Disabled
(또는 Gzip + 압축 해제 fallback 사용)

Decompression Fallback

Gzip을 쓰되 호스팅이 헤더를 못 붙일 때를 대비해 **Decompression Fallback** 체크 권장.

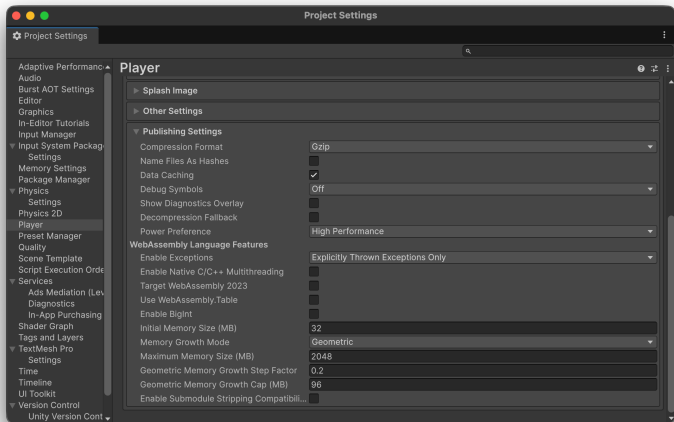
Other Settings

- **Color Space:** Gamma (기본)
- **Auto Graphics API:** 컴
- **Strip Engine Code:** 켜면 빌드 작아짐

Memory Size

프로젝트가 크면 **Initial Memory Size**를 키워야 **Out of Memory** 오류를 피할 수 있다.

Player Settings – Publishing Settings 화면



Compression Format, Decompression Fallback, Initial Memory Size 등이 모두 이 화면에 있다.

Build 실행

빌드 절차

- 1 **Build Settings** 창의
우측 하단 **Build** 버튼 클릭
- 2 출력 폴더 선택
예: C:/UnityBuilds/MyGameWeb
또는 ~/Builds/MyGameWeb
- 3 빌드 진행 (수 분 ~ 수십 분)
- 4 완료 후 **Build and Run**으로 자동 실행
가능

생성되는 폴더 구조

```
▽ MyGameWeb/  
  index.html  
  Build/  
    MyGameWeb.loader.js  
    MyGameWeb.data  
    MyGameWeb.framework.js  
    MyGameWeb.wasm  
  TemplateData/
```

주의

출력 폴더 이름은 공백/한글 없이 영문/숫자로. URL의 일부가 되기 때문이다.

로컬에서 빌드 결과 테스트

index.html을 더블 클릭하면 안 된다

file://로 직접 열면 브라우저 **CORS** 정책 때문에 .data, .wasm 파일 로딩이 실패한다.
반드시 로컬 **HTTP** 서버로 띄워서 테스트할 것.

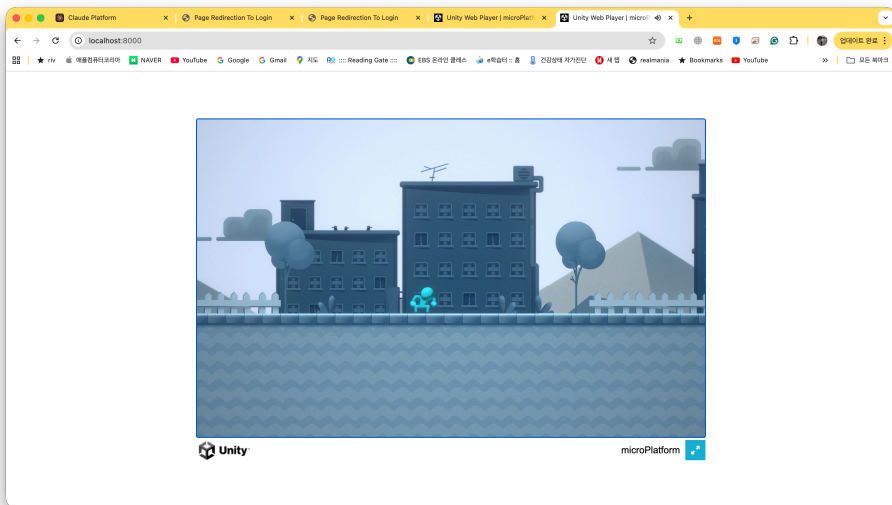
간단한 로컬 서버 (방법 1: Python)

```
1 # 빌드 출력 폴더로 이동
2 cd ~/Builds/MyGameWeb
3
4 # Python 3 내장 서버 실행 포트 ( 8000 )
5 python3 -m http.server 8000
6
7 # 브라우저에서 http://localhost:8000 접속
```

방법 2: Node.js – npx serve

```
1 cd ~/Builds/MyGameWeb
2 npx serve .
3 # 표시되는 http://localhost:3000 등으로 접속
```

브라우저에서 실행되는 WebGL 빌드



빌드 실패/오류가 났을 때

자주 발생하는 오류

- **WebGL 모듈 미설치** →
Unity Hub에서 Add modules
- **Scenes In Build 비어 있음** →
Add Open Scenes
- **경로에 한글/공백 포함** →
영문 경로로 변경
- **Out of Memory** →
Initial Memory Size 증가

디버깅 팁

- Unity 콘솔의 **빨간 오류** 먼저 해결
- 브라우저 개발자 도구(F12) -
Console 탭의 오류 메시지 확인
- Build Settings에서
Development Build 체크 →
상세 로그/프로파일러 사용 가능

여기까지 도착했다면

내 PC의 브라우저에서 `http://localhost:8000`으로 내 게임이 실행되는 상태. 이제 인터넷에 올리기만 남았다.

GitHub Pages란?

GitHub Pages란

GitHub 저장소의 내용을 그대로 정적 웹사이트로 호스팅해 주는 무료 서비스.

- HTML/JS/CSS/이미지 그대로 서빙
- 공용 URL 자동 발급:
`https://<계정>.github.io/<저장소>/`
- Unity WebGL 빌드 산출물을 그대로 올리면 동작

필요한 것

- **GitHub 계정**
(없다면 github.com 가입)
- **Git 설치**
(`git --version`으로 확인)
- 또는 **GitHub Desktop** GUI 사용

주의

업로드한 내용은 전 세계에 공개된다. 민감한 파일/키 포함 금지.

Step 1. GitHub 저장소 생성

저장소 만들기

- 1 github.com 로그인
- 2 우상단 + → **New repository**
- 3 **Repository name:** 영문/숫자
예: my-unity-webgl
- 4 **Public** 선택 (Pages 무료 사용)
- 5 **Add a README file** 체크 (선택)
- 6 **Create repository** 클릭

저장소 URL 형태

`https://github.com/
<계정>/<저장소>.git`
예:

`https://github.com/
ymkang/my-unity-webgl.git`

배포 후 URL 미리보기

`https://<계정>.github.io/
<저장소>/`

Step 2. 빌드 결과를 저장소에 올리기 (CLI)

빌드 폴더 안에서 git 초기화 후 push

```
1 # 1. WebGL 빌드 출력 폴더로 이동
2 cd ~/Builds/MyGameWeb
3
4 # 2. git 저장소로 초기화
5 git init
6 git branch -M main
7
8 # 3. 모든 파일을 스테이지에 추가
9 git add .
10 git commit -m "Initial WebGL build"
11
12 # 4. 원격 저장소 연결 앞에서 ( 만든 GitHub 주소 )
13 git remote add origin https://github.com/<MY_ID>/my-unity-webgl.git
14
15 # 5. 예) GitHub push
16 git push -u origin main
```

로그인 안내가 뜨면

GitHub는 비밀번호 대신 **Personal Access Token** 또는 **GitHub CLI / Desktop** 인증을 사용한다. GUI가 익숙하다면 **GitHub Desktop** 권장.

Step 2 (대안). GitHub Desktop 사용

GUI로 push 하기

- 1 **GitHub Desktop** 설치 후 로그인
- 2 **File** → **Add local repository**
- 3 빌드 출력 폴더 선택
- 4 “not a git repository” 안내 시 **create a repository here** 클릭
- 5 변경된 파일들이 좌측에 표시되면 **Commit to main**
- 6 상단의 **Publish repository** 클릭
(Public 체크 – Pages 사용 위해)

언제 이 방법이 좋은가

- 터미널/Git 명령이 익숙하지 않을 때
- Personal Access Token 발급이 번거로울 때
- 변경 사항을 시각적으로 확인하고 싶을 때

업로드 결과는 동일

CLI든 Desktop이든 **GitHub**의 저장소 내용만 같으면 OK.

Step 3. GitHub Pages 활성화

저장소 설정

- 1 GitHub 저장소 페이지 상단 **Settings** 클릭
- 2 좌측 사이드바 **Pages** 선택
- 3 **Source** 항목
→ **Deploy from a branch**
- 4 **Branch:** main, / (root)
- 5 **Save** 클릭

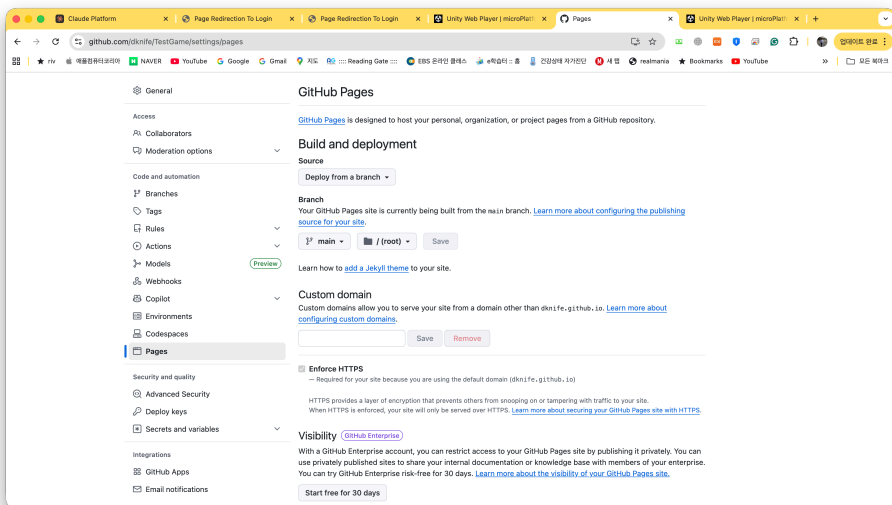
배포 URL 확인

설정 후 같은 페이지 상단에
Your site is live at
`https://<계정>.github.io/<저장소>/`
형태의 녹색 박스가 표시된다.

처음 배포는 시간이 걸린다

대개 **1~5분**.
반영이 늦으면 브라우저 **캐시 비우기** (Shift + 새로고침).

GitHub 저장소 Settings – Pages 화면



Step 4. 게임 접속 및 공유

최종 URL 형식

`https://<계정>.github.io/<저장소>/`
예) `https://ymkang.github.io/`
`my-unity-webgl/`
브라우저로 접속하면 Unity 로딩 바 →
내 게임 실행

공유 / 포트폴리오 활용

- 친구/SNS에 URL 그대로 공유
- 이력서/포트폴리오에 링크 첨부
- QR 코드로 만들어 강의실에서 시연

README.md 추천

저장소 README에
스크린샷, 조작법,
플레이 URL을 적어두면 좋다.

게임을 수정한 뒤 다시 배포하기

업데이트 워크플로우

- 1 Unity 에디터에서 수정
- 2 **File** → **Build Settings** → **Build**
– 같은 출력 폴더에 덮어쓰기
- 3 폴더(=git 저장소)에서 변경 사항 commit & push
- 4 잠시 후 GitHub Pages URL 새로고침 → 최신 버전 반영

CLI 예시

```
1 cd ~/Builds/MyGameWeb
2 git add .
3 git commit -m "Update gameplay tuning"
4 git push
```

빌드 시간이 신경 쓰인다면

Development Build는 끄고, **Strip Engine Code**는 켜 상태로 배포용 빌드를 만들면 산출물이 작아진다.

자주 발생하는 배포 문제와 해결

1. 페이지가 404로 뜸

- Pages **Source** 브랜치 확인 (main)
- index.html이 저장소 루트에 있어야 함
- URL 끝의 슬래시(/) 확인

2. 로딩 바에서 멈춤

- 브라우저 **F12** → **Console** 오류 확인
- **Compression Format**을 **Disabled**로 바꿔 재빌드
- 빌드 폴더 안의 **Build** 하위 파일 누락 여부 확인

3. “Unable to parse Build/...”

압축 헤더 문제.

- **Decompression Fallback** 체크 후 재빌드
- 또는 Compression Format = Disabled

4. 화면이 검게만 보임

- Canvas 크기 = **Default Canvas Width/Height** 확인
- URP 사용 시 **Graphics API**에 WebGL 2.0 포함되어 있는지 확인

오늘 정리

요약

- **Microgame**으로 “돌아가는 게임”을 빠르게 확보
- Unity **Build Settings** + **Player Settings**에서 플랫폼을 **WebGL**로 전환하고 빌드
- 빌드 결과 폴더를 **GitHub** 저장소로 올리고 **Pages**로 공개 URL 발급
- 수정 → 빌드 → push 만 반복하면 즉시 업데이트 배포 가능

한 줄 정리

게임은 만드는 것만큼 “보여주는 것”이 중요하다.
공개 URL이 있는 게임은 그 자체로 포트폴리오다.

과제

실습 과제

- ① Unity Microgame 또는 본인 프로젝트로 게임 준비
- ② 최소 1개 항목 이상 변경 (속도/색/문구 등)
- ③ WebGL로 빌드 – 로컬 HTTP 서버로 동작 확인
- ④ GitHub 저장소에 push, Pages로 공개
- ⑤ 제출:
 - GitHub Pages URL
 - 수정한 부분에 대한 간단한 설명 (README.md 또는 별도 보고서)
 - 게임 스크린샷 1장 이상

제출 시 주의

URL이 실제로 접속 가능한지 직접 확인 후 제출. 제출 후에도 저장소를 **Private**으로 바꾸지 말 것 (Pages가 동작하지 않게 됨).

참고 자료

공식 문서/자료

- Unity Manual – **WebGL: Building and running a WebGL project**
- Unity Manual – **WebGL: Player settings**
- Unity Learn – **Microgames**
- GitHub Docs – **Configuring a publishing source for your GitHub Pages site**
- GitHub Docs – **Quickstart for GitHub Pages**

추가로 시도해볼 만한 것

- GitHub Actions로 자동 빌드 & 자동 배포 파이프라인 구성
- itch.io, Newgrounds 등 게임 호스팅 플랫폼에도 동일 빌드 업로드
- 커스텀 도메인 연결 (Settings → Pages → Custom domain)

감사합니다

Unity로 만든 게임을 전 세계 어디서나 플레이!

강영민 - 동명대학교 게임공학과